
ジオメトリの定義

Geant4 11.3 準拠

Geant4講習会資料: HEP/NP/Space 2025



大学共同利用機関法人
高エネルギー加速器研究機構

本資料に関する注意

- 本資料の知的所有権は、高エネルギー加速器研究機構およびGeant4 collaborationが有します
- 以下のすべての条件を満たす場合に限り無料で利用することを許諾します
 - 学校、大学、公的研究機関等における教育および非軍事目的の研究開発のための利用であること
 - ・ Geant4の開発者はいかなる軍事関連目的へのGeant4の利用を拒否します
 - このページを含むすべてのページをオリジナルのまま利用すること
 - ・ 一部を抜き出して配布したり利用してはいけません
 - 誤字や間違いと疑われる点があれば報告する義務を負うこと
- 商業的な目的での利用、出版、電子ファイルの公開は許可なく行えません

目次

1. ジオメトリ定義の要点
2. Step 1: Solids(立体)の使い方
3. Step 2: Logical Volumeの使い方
4. Step 3: Physical Volumeの使い方
5. ジオメトリ定義の詳細
 1. Sensitiveの設定
 2. Volumeの識別情報 (Touchable)について
 3. 磁場空間の設定

ジオメトリ定義の要点

言葉の定義

■ ジオメトリ (*geometry*)

- シミュレーションの対象となるシステム (例: 実験室内の測定器システム、宇宙空間におかれた観測システム、など) の幾何学的な情報を一般的に *geometry* とよぶ
- システムの具体的な幾何学情報は以下のボリューム (*volume*) を単位として記述される

■ 物体あるいはボリューム (*volume*)

- システムを構成する物体の要素 (例: シリコン検出器センサ) を一般的に *volume* とよぶ
- *volume* はその幾何学形状、構成物質、配置 (据付) 位置などの情報をもつ

■ ワールド・ボリューム (*World volume*)

- システムを構成するすべての *volume* を包含する *volume*
- システム内のボリューム配置情報の基本座標系となる

[注] Geant4は右手座標系を採用している

■ 親 (*mother volume*) および子 (*daughter volume*)

- *volume* 内に他の *volume* を配置することができ、前者を *mother volume*、後者を *daughter volume* とよぶ

■ 立体 (*solid*)

- *volume* の形とサイズの情報を持つ (物質の情報はもたない)

■ 物質 (*material*)

- *volume* を構成する物質情報 (化学組成、ガス、個体等の物理状態など)

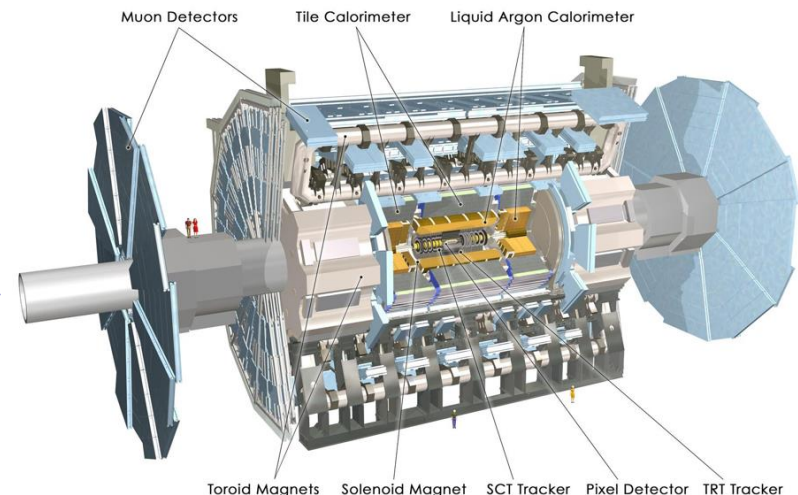
Geant4のジオメトリ定義方法を理解する鍵: *Logical Volume*

■ 複雑なシステムのシミュレーションでのジオメトリ定義の問題

- シミュレーション対象のシステムの構成エレメント数は場合によっては膨大となる
 - 例: LHC実験(CERN)での測定器はエレメント数が $\sim 10^8$ のオーダー
- それぞれのエレメントを独立なvolumeで記述すると、必要メモリのサイズが膨大となり、メモリへのアクセスが処理速度のボトルネックとなる
- 従ってメモリを効率利用できるジオメトリ定義方法が重要

■ Geant4ジオメトリ定義法の基本設計思想

- 構成エレメント数が膨大なシステムでは。多くの場合、同じエレメントが繰り返し使われる
 - 例: Atlas/LHC実験のPixel検出器
- 同一のエレメントが多数ある場合、エレメントを一つのvolumeとして定義し、それを共有することでメモリ消費を軽減できる
- Geant4ではこの共有volumeを*Logical Volume*とよび、ジオメトリ定義の基本となる



[要点]

Geant4ジオメトリ定義では対象エレメントに対して、必ず*Logical Volume*を作り、それをWorld volumeあるいは他のlogical volumeに配置する手法をとる

ジオメトリ定義の流れ

ジオメトリ定義にはGeant4が提供するジオメトリ・クラスを用いる

← 基本的な定義ステップは以下のとおり：（コード例は次のスライドに示す）

■ ステップ 1: Solid定義

- Geant4が用意している立体(G4VSolid)の中から適当な形状を選択し、そのサイズを設定

[例] 直方体: $2 \times 3 \times 5 \text{ m}^3$; 円筒: 直径=1m、長さ=3m; など

■ ステップ 2: Logical Volume定義

- 上の立体に物質情報、有感情報、電磁場情報などを付加し、ボリューム(Volume)を作る
- このボリュームはシステムのステップ3で繰り返し使うことができるもので Logical Volume (G4LogicalVolume)とよばれる
- Logical Volumeは空間配置されていないので、位置情報は持たない

[例] 直方体のシリコン素子

■ ステップ 3: Physical Volume定義

- 上で作られたLogical Volumeを物理的に空間に配置する
- 配置されたボリュームはPhysical Volume (G4PhysicalVolume)とよばれる

[例] 実験室の中心に配置された直方体シリコン素子

ジオメトリ・クラスの使用例

■ ジオメトリ・クラスを使ったジオメトリ定義の具体例（各クラスの詳細は後述）

Step 1. Solid定義： 形状およびサイズを指定

```
G4VSolid* pBoxSolid =  
    new G4Box{ "aBoxSolid", 1.*m, 2.*m, 3.*m };
```

↑ Box形状を作る

Step 2. Logical Volume定義： solidに物質情報などを付加

```
G4LogicalVolume* pBoxLog =  
    new G4LogicalVolume{ pBoxSolid,  
        pBoxMaterial, "aBoxLog", 0, 0, 0 };
```

↑ 物質情報（作製法は後述）

Step 3. Physical Volume定義： logical volumeに位置、回転情報などを付加して親ボリウムに配置

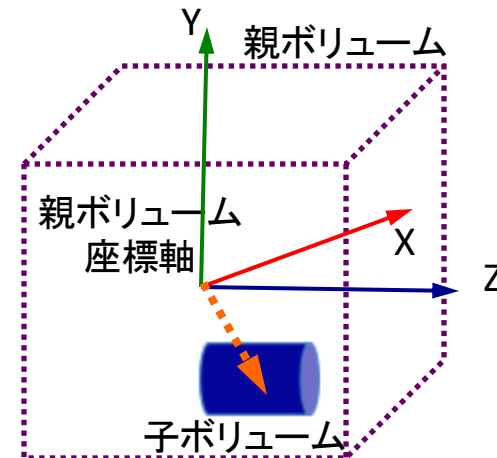
```
G4VPhysicalVolume* aBoxPhys =  
    new G4PVPlacement{ pTrans3D, pBoxLog, "aBoxPhys", pMotherLog, 0, copyNo };
```

↑ 位置・回転情報（作製法は後述）

↑ 親ボリウムの指定

[注]

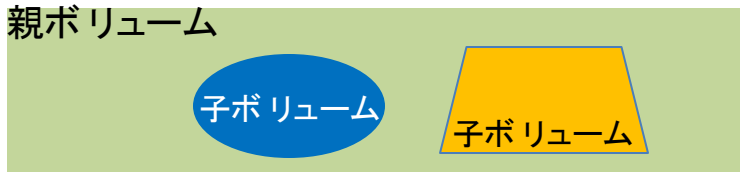
- ボリウム (Volume)は親ボリウム(Mother Volume)の中に配置される ← Physical Volumeを作るともいう
- 配置を表す回転、位置情報は親物体の固有座標系を用いて表現される
- 親物体の固有座標系は親物体を構成するsolidの座標系に一致
- 配置された物体は子ボリウム(Daughter Volume)ともよばれる



Logical Volume配置における留意事項

- 子ボリュームを親ボリューム内に配置すると、置かれた部分のボリューム情報(物質情報など)は子ボリュームのものに置き変わる

親ボリューム



[注]

子ボリュームは必ずlogical volumeでなければならないが、親ボリュームはlogical又はphysical volumeでOK

- 子ボリュームを親ボリュームから飛び出して配置することは禁止



- ボリューム同士を交差して配置することは禁止



- ボリューム同士を接して配置することはOK



[注]

接しているか、交差しているかの判断は位置情報の数値表現精度で判断される

ユーザが作成したジオメトリが、左記の禁止事項に抵触するかを
チェックツールを使うことができる

➡ Appendix 1 を参照

ジオメトリ定義の作成

■ 基本的な手続きは以下のようになる:

- 1) G4VUserDetectorConstructionクラスを継承してユーザのジオメトリ定義クラスを作成
- 2) そのジオメトリ・クラスのConstruct関数内にユーザのジオメトリを記述
- 3) 完成したジオメトリ定義クラスのオブジェクトを作成し、G4RunManagerに登録

```
//+++++ヘッダーファイル+++++
// Geometry.hh
//+++++
#ifndef Geometry_h
#define Geometry_h 1

#include "G4VUserDetectorConstruction.hh"
class G4VPhysicalVolume;

//-----
class Geometry : public G4VUserDetectorConstruction
//-----
{
public:
    Geometry();
    ~Geometry();

    G4VPhysicalVolume* Construct();

private:
    G4VPhysicalVolume* ConstructDetector();
};
#endif
```

ユーザのジオメトリ
定義クラス
(名前は任意)

Geant4が提供する
ジオメトリ定義用の
基底クラス

ジオメトリ定義の記
述はこの関数に書く

実装の詳細は
Hands-Onで学ぶ

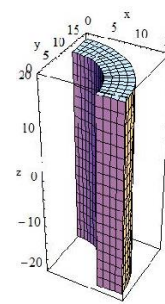
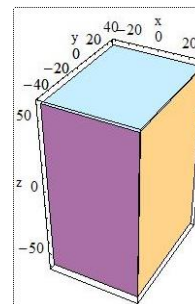
Step 1: Solids(立体)の使い方

Solidの定義 – Geant4が提供する立体

■ Geant4では立体(*solid*)を表現するのに以下のタイプを使用する:

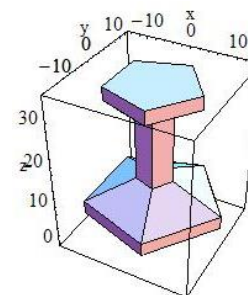
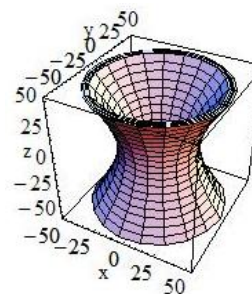
- CSG(Constructive Solid Geometry)立体

- G4Box, G4Tubs, G4Cons, G4Trd, ...
- 単純な形状の立体表現



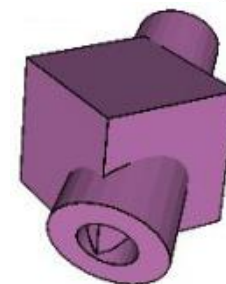
- 特殊形状立体 (CSGと同じ性質)

- G4Polycone, G4Polyhedra, G4Hype, ...



- Boolean(ブール演算)で表現出来る立体

- G4UnionSolid, G4IntersectionSolid, G4SubtractionSolid

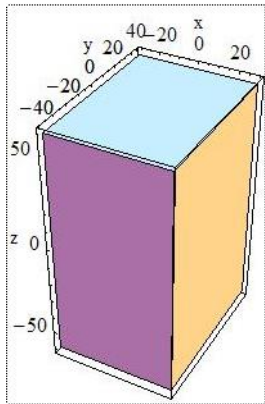


- 多数の切子面で出来たCAD向け形状

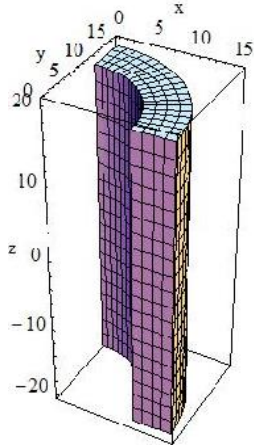
- G4TesselatedSolid



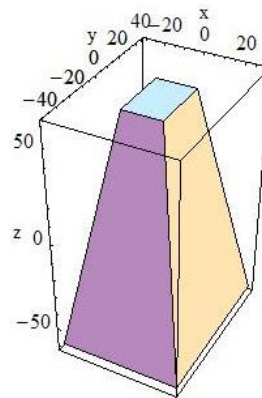
Solidの定義 – CSG立体の種類



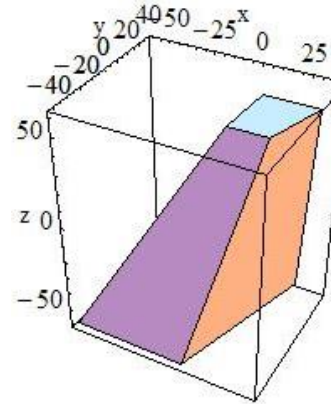
G4Box



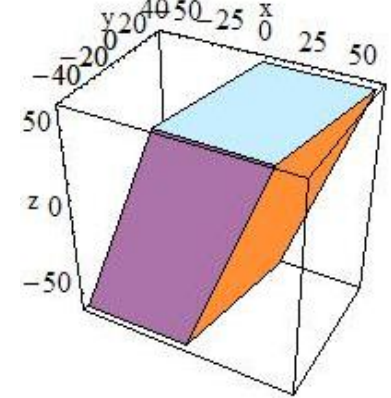
G4Tubs



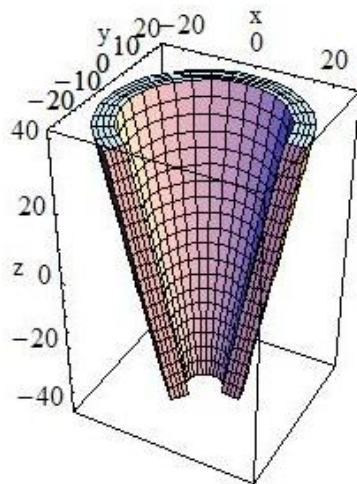
G4Trd(trapezoid)



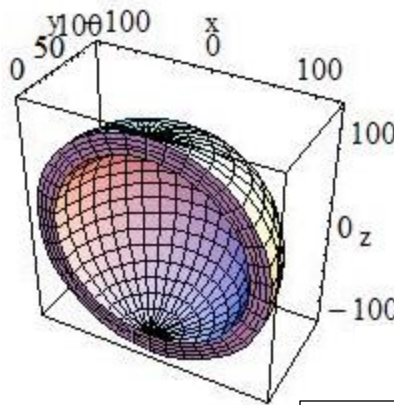
G4Trap (generic
trapezoid)



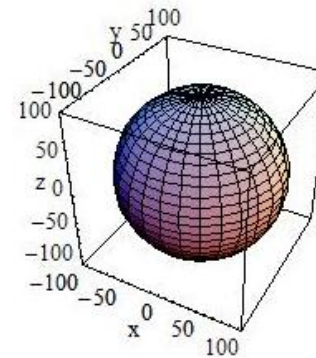
G4Para
(parallelepiped)



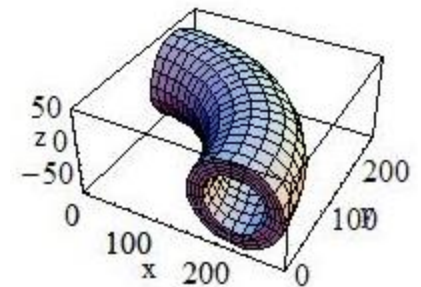
G4Cons



G4Sphere



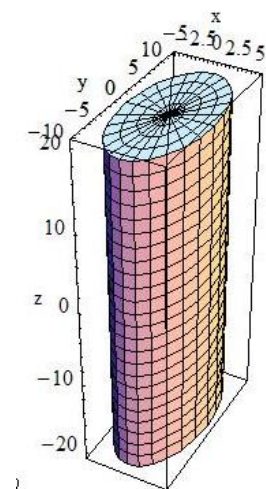
G4Orb (中空でない球体)



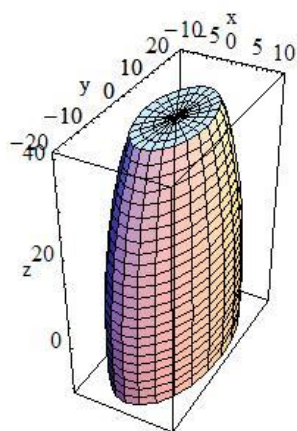
G4Torus

ユーザマニュアル 'Book For Application Developers' の 以下の章を参照:
Detector Definition and Response -> Geometry -> Solids

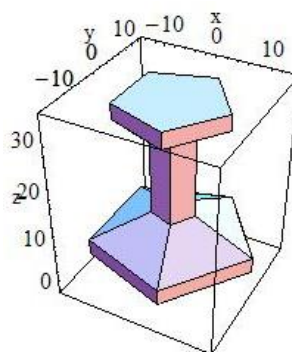
Solidの定義 - 特殊形状CSG立体



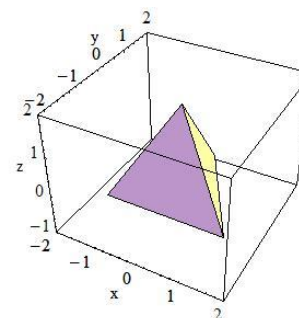
G4EllipticalTube



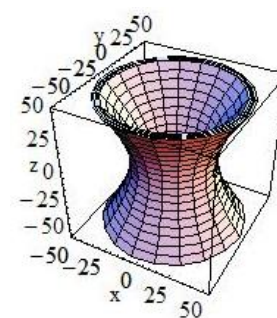
G4Ellipsoid



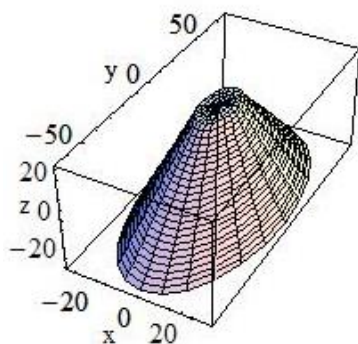
G4Polyhedra



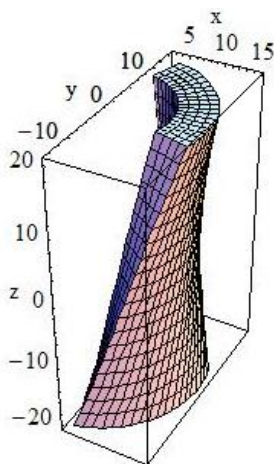
G4Tet
(tetrahedra)



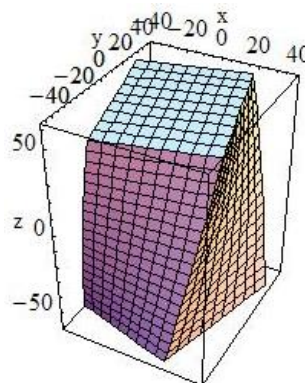
G4Hype



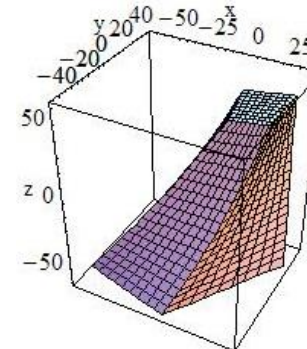
G4EllipticalCone



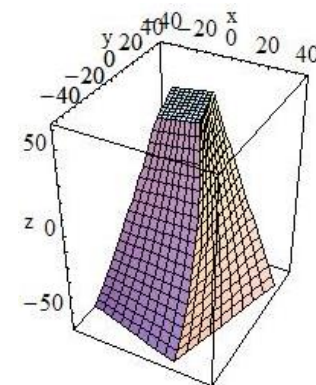
G4TwistedTubs



G4TwistedBox



G4TwistedTrap



G4TwistedTrd

ユーザマニュアル 'Book For Application Developers' の 以下の章を参照:
Detector Definition and Response -> Geometry -> Solids

Solidの定義 – ブール表現

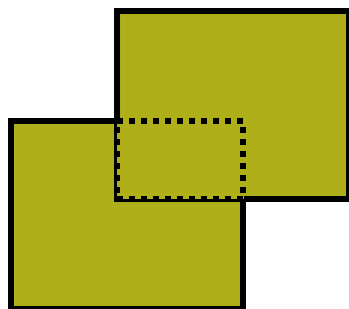
■ 立体をブール演算で表現することができる

- **G4UnionSolid**, **G4SubtractionSolid**, **G4IntersectionSolid**
- 2つの立体とそれに対する上記ブール演算を指定してブール立体を表現
- 2番目の立体は1番目の立体の空間座標系に対して用いて配置される
- ブール演算で得られるのは一つの立体(ブール立体)であるから、必要に応じて3番目の立体を演算させるが可能

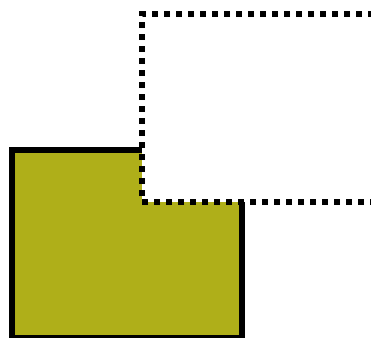
■ 演算に使える立体は基本的にCSGかあるいはブール立体である

[注] ブール立体はCSG立体に比べて粒子trackingの時間が長くなる可能性がある

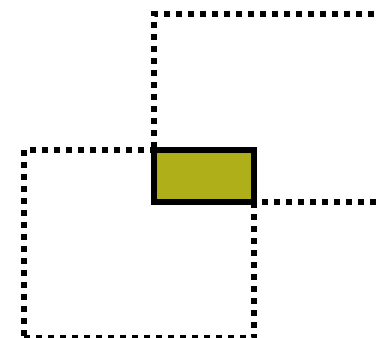
G4UnionSolid



G4SubtractionSolid



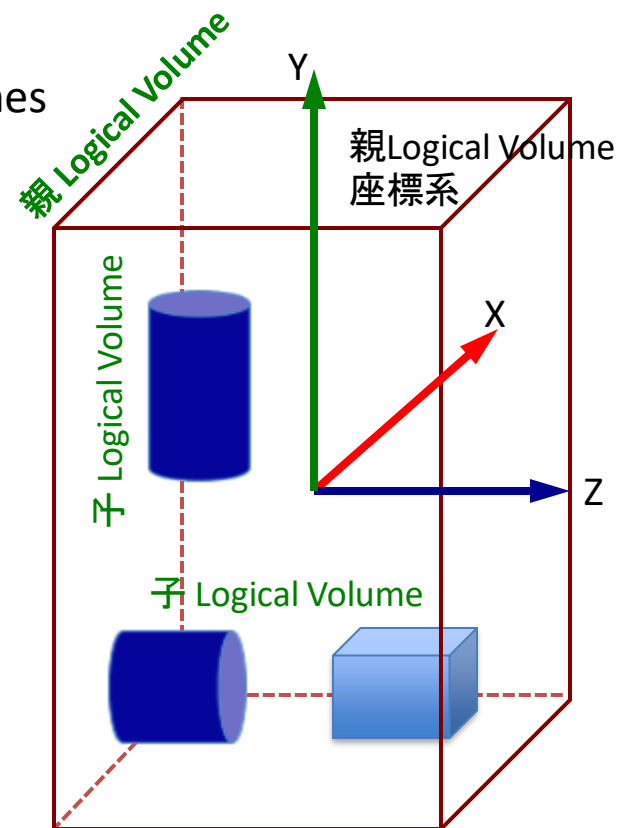
G4IntersectionSolid



Step 2: Logical Volumeの使い方

Logical Volumeの定義 – Volumeの階層性

- *Logical Volume*で定義された物体は使い回しが可能
- 一つのlogical volume (親物体) の中に複数のlogical volumes (子物体) を配置できる
- 親物体は子物体の*G4LogicalVolume*へのポインタを保持
 - ただし、子物体は親物体の情報(ポインタ)を保持しない
 - 親物体を他の物体の中に配置すると、親物体に含まれている物体(子物体)も同時に配置される
- **ワールド**(*World Volume*)は親子関係の最上位にあり、他の全ての物体は包含していなければならない
 - ‘**ワールド**’が保持する座標系は*global coordinate system* (グローバル座標系)とよばれ、ジオメトリの全体の基本座標系である
- 粒子の位置情報は*global coordinate system*で表現される
 - 位置情報を子物体の座標系に変換する手法は用意されている



[注]

子volumeが配置されることで、それが占める親volumeの部分は子volumeに置き換わる

Logical Volumeの定義 – G4LogicalVolumeクラス

```
G4LogicalVolume(G4VSolid* pSolid,  
                G4Material* pMaterial,  
                const G4String &name,  
                G4FieldManager* pFieldMgr=0,  
                G4VSensitiveDetector* pSDetector=0,  
                G4UserLimits* pULimits=0,  
                G4bool optimise=true);
```

■ G4LogicalVolumeオブジェクトが保持する情報

- 形状とサイズ (G4VSolid) : 設定は**必須**
- 物質情報 (G4Material) : 設定は**必須** (volume中で粒子をtrackingさせない場合を除く)
- 名前 (G4String) : 設定は**必須**ではないが値を入れることを推奨
- 電磁場情報 (G4FieldManager)
- 有感度設定 (G4VSensitiveDetector)
- ユーザ設定条件 (G4UserLimits)
- ジオメトリの最適化手法のon/off (G4bool)

これらの引数は後の講義で説明があるので、現時点では無視してOK

← デフォルト値が設定されているので、引数を設定しなくてもLogical Volumeは定義できる

[注]

- Volumeをグラフィックス上で表示するかどうかは、SetVisAttributes()関数で設定できる
- G4LogicalVolumeは基底クラスとして使用は**不可** (仮想クラスではない)

Logical Volumeの定義 – 物質の表現

- Logical volumeを定義するには物質を表現する以下のクラスが必要
- G4Isotope: 一つのアイソトープを表現するクラス
 - 原子番号、質量数、モル質量、など
- G4Element: 一つの元素を表現するクラス
 - (有効)原子番号、(有効)質量数、(有効)原子量、シェル数、など
 - 含有アイソトープの情報(存在比: 天然存在度等)
 - Radiation length、イオン化パラメター、など
- G4Material: 物質の化学的・物理的性質を表現するクラス
 - 化学式、構成元素の数、質量比、など
 - 密度、状態(固体、液体、ガス)、温度、など
 - Radiation length, nuclear interaction length,など

[注]

シミュレーションの幾何学形状の定義で必要となるのはG4Materialのみ

➡ G4Element/G4IsotopeはG4Materialを作るときだけに使用される

Logical Volumeの定義 – 物質の表現 (つづき)

- ユーザが元素、物質等を独自に定義することもできるが、通常はGeantが標準提供する**NIST 物質データベース**(National Institute of Standards and Technology, U.S.)**を使うことを奨励する**
- NISTデータベースの使い方

```
G4NistManager* manager = G4NistManager::GetPointer();  
  
G4Element* elm = manager->FindOrBuildElement("atomic-symbol", G4bool iso);  
G4Element* elm = manager->FindOrBuildElement(G4int iZ, G4bool iso);  
G4Material* mat = manager->FindOrBuildMaterial("name", G4bool iso);  
G4double isotopeMass = manager->GetMass(G4int iZ, G4int N);
```

- G4bool iso = true : 元素組成は自然界のアイソトープ比を用いる
- G4bool iso = false : アイソトープを用いない

Logical Volume定義のコード例

- 以下に空気で満たされたWorld Volume(本節の最初のスライド参照)の logical volumeを定義するコードを示す

```
// Get pointer to 'Material Manager'
```

```
G4NistManager* materi_Man = G4NistManager::Instance();
```

← NISTマテリアル
マネージャの作成

```
// Define the shape of solid
```

```
G4double leng_X_World = 2.0*m;           // X-full-length of world
```

```
G4double leng_Y_World = 2.0*m;           // Y-full-length of world
```

```
G4double leng_Z_World = 2.0*m;           // Z-full-length of world
```

```
auto solid_World =
```

```
    new G4Box{ "Solid_World", leng_X_World/2.0, leng_Y_World/2.0, leng_Z_World/2.0 };
```

← Worldのsolidを作成

```
// Define logical volume
```

```
G4Material* materi_World = materi_Man->FindOrBuildMaterial( "G4_AIR" );
```

← NISTから空気を取得

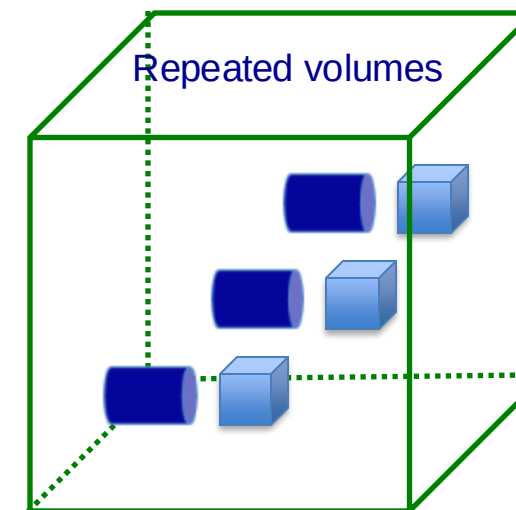
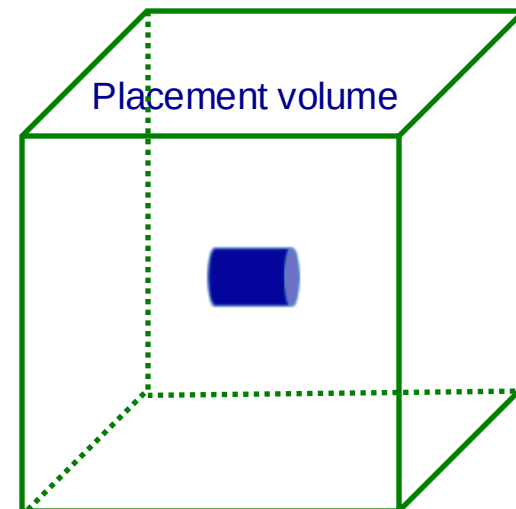
```
auto logVol_World = new G4LogicalVolume{ solid_World, materi_World, "LogVol_World" };
```

← Worldのlogical
volumeを作成

Step 3: Physical Volumeの使い方

Physical Volumeとは

- **Physical Volume**とは親volume内に配置した、子logical volumeをさす
 - 子volumeが置かれた部分のvolume情報(物質など)は親volumeでなく、子volumeのものに置きかわる
 - 親volumeは子volumeの位置、回転の情報を保持
 - 親volumeは**logical volume**又は**physical volume**のいずれでも良い
- 配置には以下の制限がある:
 - 子物体は親物体からはみ出してはいけない
 - 子物体同士は互いに重なり合ってはいけない
- Physical Volumeを作る: **大きく分けて二つの手法**
 - **Placement volume**: 親物体空間に一度だけ物体を置く
 - 一つのphysical volumeオブジェクトは、親物体空間に配置された一つの物体に対応している
 - **Repeated volumes**: 親物体空間に複数回、物体を置く
 - 一つのphysical volumeオブジェクトが親物体空間に置かれた任意の個数のvolumesに対応する
 - 以下の手法がある:関数表現をすることで柔軟に物体配置ができる
 - レプリカ
 - パラメーター表現 (*parameterization*)
 - この表現を用いることでメモリーを節約できる



G4PVPlacement

■ G4PVPlacementのコンストラクターは以下のとおり

```
G4PVPlacement(  
    const G4Transform3D &Transform3D,           // 子物体配置の回転・並進情報を表すオブジェクト  
    G4LogicalVolume *pLogical,                   // 子物体のlogical volumeへのポインタ  
    const G4String &pName,                       // 配置された後の子物体の名前(配置の名前)  
    G4LogicalVolume *pMotherLogical,            // 親物体(配置先)logical volumeへのポインター  
    G4bool pMany,                               // この引数は現在、使われていない  
    G4int pCopyNo,                              // コピー番号([注 1]参照)  
    G4bool pSurfChk=false);                    // 物体が重なって配置されているかのチェック([注 2]参照)
```

[注 1] コピー番号について

- 整数値を設定することで、G4PVPlacementによるlogical volume配置にコピー番号(配置したlogical volumeのID)を付加できる ← 使わないなら全てゼロとしてOK
- コピー番号は重複を許すが、配置IDとして使用したいならば異なった値にする

[注 2] 物体が重なって配置されているかのチェック

- 物体を重ねて配置することは禁止: フラグを'true'にすれば、重なりの有無を自動的にチェックしてくれる(trueに設定を推奨)

[注 3] G4PVPlacementのコンストラクタの種類

- 上に示したコンストラクタ以外に幾つかのシグネチャーの異なるコンストラクタがある
- 子物体を親物体のphysical volumeに配置したい場合、以下のコンストラクタを使う (ワールドへの配置等)

```
G4PVPlacement(  
    const G4Transform3D &Transform3D,           // 子物体配置の回転・並進情報を表すオブジェクト  
    const G4String &pName,                     // 配置された後の子物体の名前(配置の名前)  
    G4LogicalVolume *pLogical,                 // 子物体のlogical volumeへのポインタ  
    G4VPhysicalVolume *pMotherPhys,           // 親物体(配置先)physical volumeへのポインター  
    G4bool pMany,                             // この引数は現在、使われていない  
    G4int pCopyNo,                            // コピー番号([注 1]参照)  
    G4bool pSurfChk=false);                  // 物体が重なって配置されているかのチェック([注 2]参照)
```

- Geant4の公式examplesでは以下のコンストラクタがよく使われる

G4PVPlacement(G4RotationMatrix *pRot, const G4ThreeVector &tlate, G4LogicalVolume.....)

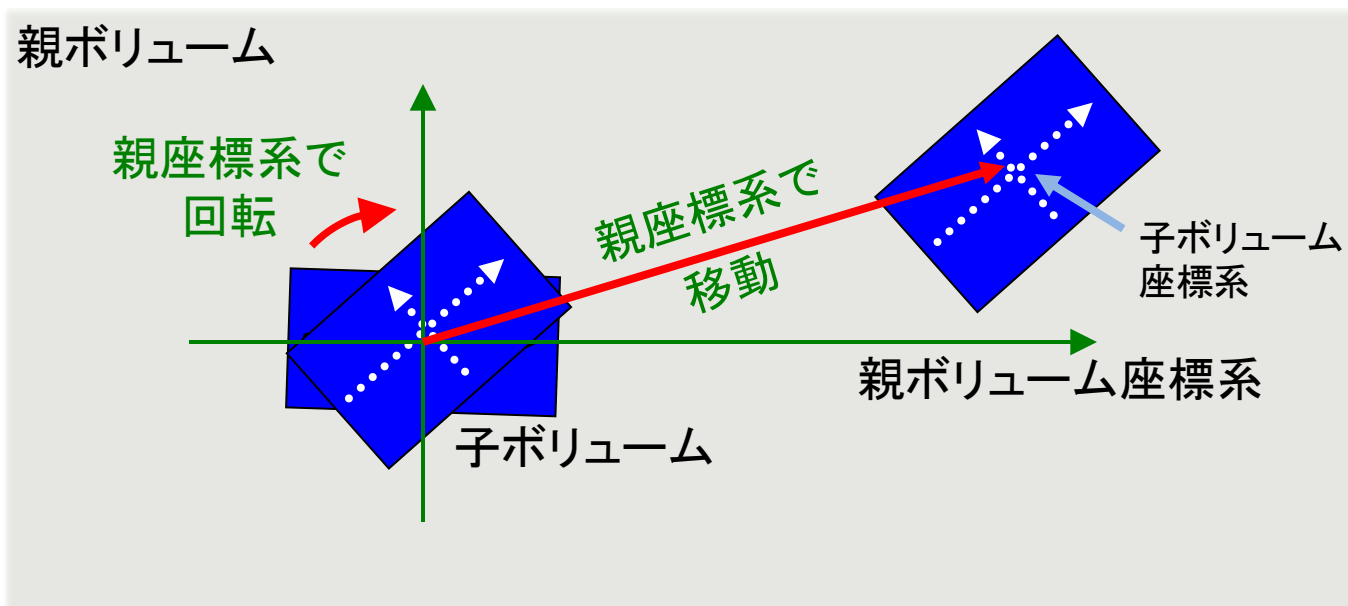
しかし、このコンストラクタは回転配置の表現で混乱を招く恐れがある → 上のコンストラクタ使用を推奨

Physical Volumeの作成

■ G4PVPlacement(前スライドのコンストラクタ)を使った場合のvolume配置表現

- 子物体を親座標系で回転
- 親物体の座標系で子物体を空間移動(並進)

[注] 回転と移動は独立であるから操作順序は考えなくて良い



G4Transform3Dについて

- *G4Transform3D*は物体の3次元座標系での回転と並進移動を表現するクラス
 - 回転は*G4RotationMatrix*クラスを用いて表現
 - 並進移動は*G4ThreeVector*クラスを用いて表現
 - CLHEP (CERNが開発したHEP用のC++ライブラリ) の*Transform3D*クラスの'typedef'

```
typedef HepGeom::Transform3D G4Transform3D;
```

- ◆ *Transform3D*クラスの定義ファイルはGeant4配布コードの以下にある:

Geant4/externals/clhep/include/CLHEP/Geometry/Transform3D.h

- *G4RotationMatrix*は物体の3次元座標系での回転を表現するクラス

- CLHEPの*HepRotation*クラスの'typedef'

```
typedef CLHEP::HepRotation G4RotationMatrix;
```

- ◆ *HepRotation*クラスの定義ファイルはGeant4配布コードの以下にある:

Geant4/externals/clhep/include/CLHEP/Vector/Rotation.h

- *G4ThreeVector*は3次元ベクトルを表現するクラス

- CLHEPの*Hep3Vector*クラスの'typedef'

```
typedef CLHEP::Hep3Vector G4ThreeVector;
```

- ◆ *Hep3Vector*クラスの定義ファイルはGeant4配布コードの以下にある:

Geant4/externals/clhep/include/CLHEP/Vector/ThreeVector.h

[注] これらのクラスの使用例を次に示すが、使い方の詳細は上に示した定義ファイルを参照

G4Transform3D・G4RotationMatrix・G4ThreeVectorの使用例

■ *G4ThreeVector* (#include "G4ThreeVector.hh")

```
G4ThreeVector v1{1.0, 2.0, 3.0};  
G4cout << v1.x() << ", " << v1.y() << ", " << v1.z() << G4endl;  
G4cout << v1.theta() << ", " << v1.phi() << ", " << v1.mag() << G4endl;  
G4ThreeVector v2{};  
v2.setX(10.); v2.setY(20.); v2.setZ(30.);  
G4cout << v2.x() << ", " << v2.y() << ", " << v2.z() << G4endl;
```

← コンストラクタ
← x, y, z 値
← theta, phi 値と絶対サイズ
← コンストラクタ
← x, y, z 値 設定

■ *G4RotationMatrix* (#include "G4RotationMatrix.hh")

```
G4RotationMatrix rot1{};  
rot1.rotateY(10.0*deg); // rotation in Y: 10.0 deg  
rot1.rotateY(20.0*deg); // rotation in Y: 20.0 deg (additional)  
  
G4RotationMatrix rot2{};  
rot2.rotateY(10.0*deg); // rotation in Y: 10.0 deg (first in Y-axis)  
rot2.rotateZ(20.0*deg); // rotation in Z: 20.0 deg (then in Z-axis)  
  
G4RotationMatrix rot3{};  
rot3 = rot1 * rot2; // multiply  
rot3 = G4RotationMatrix{}; // reset 'rot3'  
  
G4cout << rot1.xx() << ", " << rot1.xy() << ", " << rot1.xz() << G4endl;  
G4cout << rot1.yx() << ", " << rot1.yy() << ", " << rot1.yz() << G4endl;  
G4cout << rot1.zx() << ", " << rot1.zy() << ", " << rot1.zz() << G4endl;
```

← コンストラクタ
← y 軸で連続2回転するマトリックス
← コンストラクタ
← y 軸の次に Z 軸で回転するマトリックス
← コンストラクタ
← 二つのマトリックスの積
← rot3 マトリックスの値をリセット
← rot1 マトリックスの各成分

■ *G4Transform3D* (#include "G4Transform3D.hh")

```
G4Transform3D trans(rot1, v1); // transform3D using rot1 and v1
```

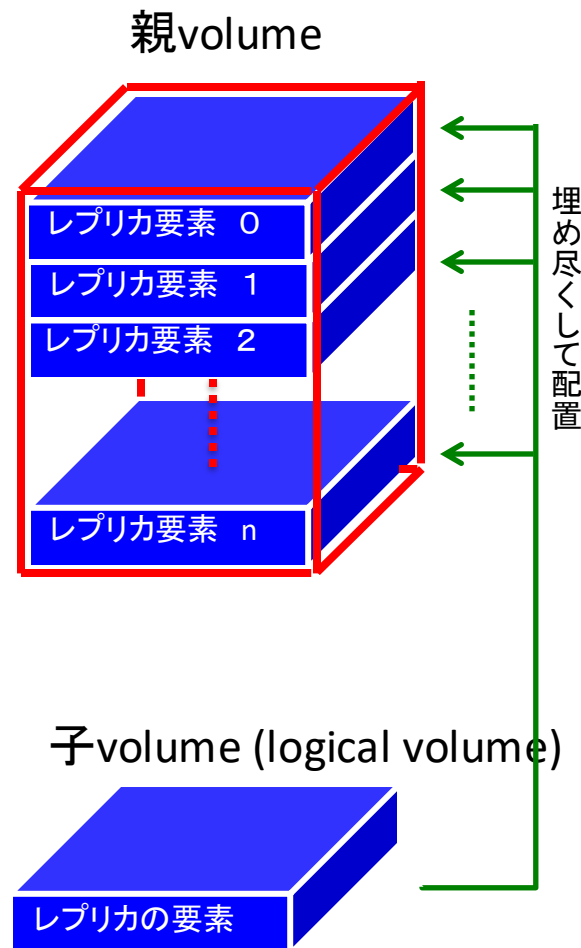
← コンストラクタ

レプリカ (Replicated Volume) とは

- 先の "G4PVPlacement" と同じで、"G4PVReplica"
(レプリカ) はレプリカ要素を親volumeに配置する手法
- "G4PVPlacement" がvolumeを一つずつ配置するのに対し、
レプリカはレプリカ要素volumeを一度に複数配置できる
- 親物volume内部を同一のサイズ・形状を持つレプリカ要素
で完全に埋め尽くしたい時に使う
- 多量の配置ではレプリカを使えばメモリー節約量が多い

[注]

- レプリカの親volumeはlogical volume, physical volumeの
いずれでも良い
- レプリカ要素volume内にG4PVPlacementでvolumeを配置可能
➡ このvolumeは親volumeや他のレプリカ要素と交差／重複
することは不可
- パラメータ表現立体 (Parameterised volumes、後述) をレプリカ
要素内に配置することは不可
- それぞれのレプリカ要素にはゼロから始まる連続番号(replica
number) が自動的に振られる



G4PVReplicaクラス

■ レプリカを表現するクラス

```
G4PVReplica(const G4String &pName,
```

```
G4LogicalVolume *pLogical,
```

```
G4LogicalVolume *pMother,
```

```
const EAxis pAxis,
```

```
const G4int nReplicas,
```

```
const G4double width,
```

```
const G4double offset=0.);
```

子volume (レプリカ)

親volume: **G4VPhysicalVolume***にすることも可能

配置方向

配置するレプリカの数

配置するレプリカの配置方向の厚み

配置を開始場所の原点からのオフセット値

[使用例]

■ X軸方向への配置

```
G4PVReplica repX("Linear Array", pRepLogical, pContainingMother, kXAxis, 5, 10*mm);
```

■ 動径方向への配置

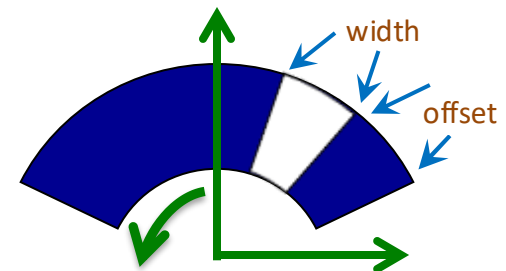
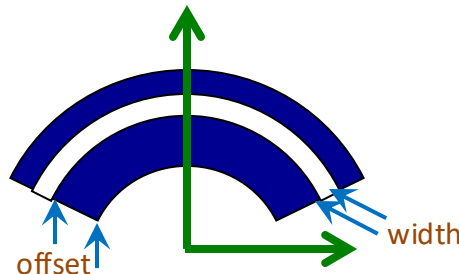
```
G4PVReplica repR("RSlices", pRepRLogical, pContainingMother, kRho, 5, 10*mm, 0);
```

■ Z軸方向への配置

```
G4PVReplica repRZ("RZSlices", pRepRZLogical, &repR, kZAxis, 5, 10*mm);
```

■ 極軸方向への配置

```
G4PVReplica repRZPhi("RZPhiSlices", pRepRZPhiLogical, &repRZ, kPhi, 4, M_PI*0.5*rad, 0);
```



その他のVolume配置方法

先に述べたPVPlacementとReplica以外に、以下の配置方法が使える

■ パラメータ配置: [G4VPVParameterisation](#)

- レプリカを発展させたもので、配置される要素ごとに形状、サイズ、物質を変えることができる

■ ネステッド パラメータ配置: [G4VNestedParameterisation](#)

- パラメータ配置の発展したもので、配置される要素の中にreplica, パラメータ配置を入れ子にできる

■ レプリカ スライス: [G4ReplicatedSlice](#)

- レプリカを発展させたもので、配置される要素厚み、間隔、オフセットを設定できる

[注] これらのクラスの詳細は”Book for Application Developers”を参照

ジオメトリ定義の詳細:

Sensitive Detectorの設定

Sensitive Detector

■ Sensitive Detectorとは

- 一般的にジオメトリ定義で作るボリュームは以下の二種類に分類できる
 - ・ 検出器のように粒子が入ると何らかの情報・信号を返すもの → **active volume**
 - ・ 粒子が入っても情報・信号を返さないもの → **passive volume**
- **Sensitive Detector**(*G4VSensitiveDetector*)は**active volume**を表現するために用いる
 - ・ **logical volume**にsensitive detectorを表現するオブジェクトへのポインターを設定する
 - ・ 粒子がこのボリュームに入射すると、sensitive detectorオブジェクトが持つ関数がよばれ、その関数で定義されている処理をおこなう

■ G4VSensitiveDetectorクラス

- 基本的にはG4UserSteppingActionと同じで、ユーザがtrackingの各ステップごとに何らかの処理をしたい場合に使う
 - ・ G4SteppingActionとの違い: 粒子がsensitive detector内にいる時だけよばれる

使用例はHands-Onで学ぶ

ジオメトリ定義の詳細:

Volumeの識別情報 (Touchable) について

Touchableとは

■ ジオメトリ定義ではvolumeが再利用がされ、親volumeに配置される

- volumeに付加されている名前、copy/replica番号のみでは、volumeがgeometry treeのどこに位置するかを特定できない
- 位置特定のためには追加情報が必要
- この追加情報をtouchableとよぶ

[注] 'Touchable'という言葉は馴染みないであろうが、歴史的にGeant4で使われてきている。
Logical volume等が物理的には触れないのに対して、'Touchable'は接触可能という程度の意味

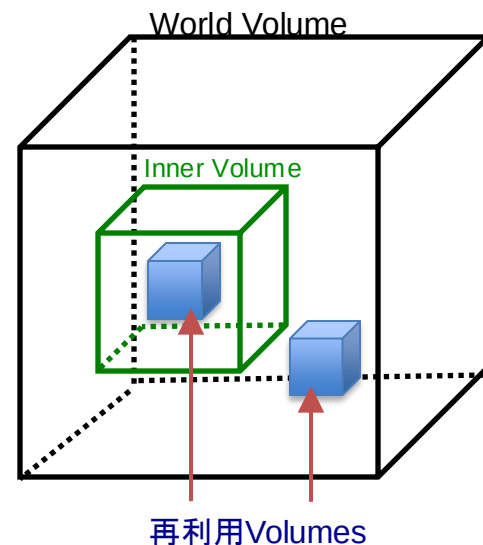
■ touchableの特性

- volumeのgeometry tree内での位置情報を持つ
- リクエストに応じて動的に作成される
- G4TouchableHistoryクラスのオブジェクトで表現されている

[注] Geant4 11.2以後、これが唯一の表現クラス（以前は複数あった）

■ touchableオブジェクトから以下の情報が取得できる

- volumeのworld volumeにいたる階層関係(volume history)
- 各階層のvolumeへのポインター
- World volume座標系と各階層のvolume座標系との変換を表現する並進・回転マトリクス
- Replica volumeの場合、そのreplica番号



Touchableのユースケース

■ ユーザがtouchableを必要とする代表的なユースケース

- 粒子のいるvolumeの上位階層にあるvolumeの情報を知りたい
- World volume座標系で与えられる粒子の位置を、volumeのlocal座標系に変換したい
- 粒子のいるvolumeがReplicaで定義されている時、そのreplica numberが知りたい

■ ユーザのtouchable入手の方法:

- Touchable入手には粒子のStepPointが持つ関数GetTouchableHandle()を使用

```
G4StepPoint* prePoint = step->GetPreStepPoint();  
G4TouchableHandle touch = prePoint->GetTouchableHandle();
```

- 関数が返す'touch'はtouchableへのポインタ: "->"を使ったアクセスが必要

```
G4VPhysicalVolume* volume = touch->GetVolume();
```

[注] G4TouchableHandle とは

- G4TouchableHandleはG4TouchableHistory(touchableを表現するクラス)の"ハンドル"
- ハンドルは作られると、操作対象のオブジェクト(今の場合、G4TouchableHistory)を動的に生成し、そのポインタを返すとともに、オブジェクトの解放も管理(スマートポインタ)
- ユーザはハンドルを使うことでメモリ管理を気にせずに対象のオブジェクトにアクセスできる

具体的なtouchableの使い方は次のスライドを参照

Touchableの使い方

■ G4TouchableHandleでよく使うG4TouchableHistoryの関数

<code>G4VPhysicalVolume* GetVolume(G4int depth = 0);</code>	← 各階層Volumeへのポインタ
<code>G4VSolid* GetSolid(G4int depth = 0);</code>	← 各階層Solidへのポインタ
<code>G4ThreeVector& GetTranslation(G4int depth = 0);</code>	← 各階層Volumeへの座標変換 (並進)
<code>G4RotationMatrix* GetRotation(G4int depth = 0);</code>	← 各階層Volumeへの座標変換 (回転)
<code>G4int GetReplicaNumber(G4int depth = 0);</code>	← 各階層VolumeがReplicaの時のreplica number
<code>G4int GetCopyNumber(G4int depth = 0);</code>	← 各階層VolumeがPVPlacementの時のcopy number
<code>G4int GetHistoryDepth();</code>	← 全階層数

■ G4TouchableHistoryの関数使用例

```
// Touchable Handle入手
G4StepPoint* prePoint = step->GetPreStepPoint();
G4TouchableHandle touch = prePoint->GetTouchableHandle();

// 粒子のいるVolume(0階層)からworld volumeへの階層数入手
G4int tDepth = touch->GetHistoryDepth();

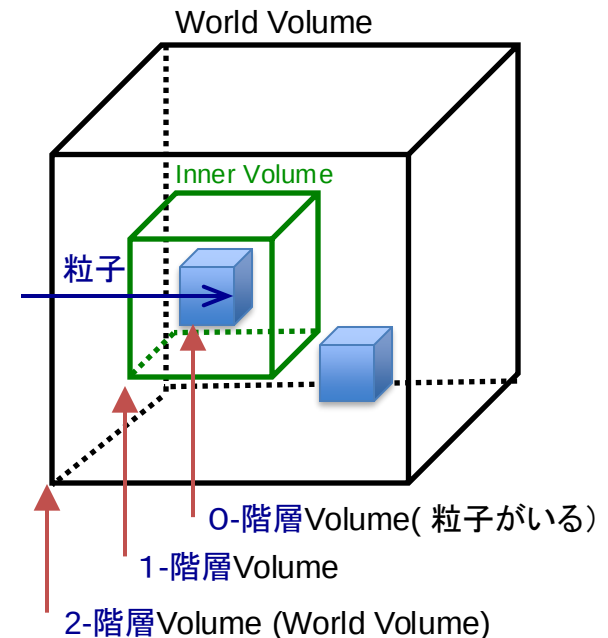
// i番目階層Volumeへのポインタ入手 (iは0から)
G4VPhysicalVolume* volume = touch->GetVolume(i);

// i番目階層Volumeのcopy/replica number入手
// [注] volumeがPVPlacementかReplicaかで関数を使い分ける
G4int copyNum = touch->GetCopyNumber(i); // PVPlacementの場合
G4int replNum = touch->GetReplicaNumber(i); // Replicaの場合

// 粒子位置情報(world座標系)を粒子のいるVolume座標系に変換
G4ThreeVector worldPosition = prePoint->GetPosition();
G4ThreeVector localPosition = touch->GetHistory(0)->
    GetTopTransform().TransformPoint(worldPosition);
```

[注]

- 粒子のいるvolumeのmaterial情報はtouchableではなく、StepPointから入手すること: `step->GetMaterial();`
- Touchableから入手したVolumeポインタを使い、volume情報を取得する場合、取得値を十分に検証すること



ジオメトリ定義の詳細:

磁場空間の設定

磁場の表現

- 磁場の表現は *G4MagneticField* を継承したクラスを実装しておこなう

- 一様磁場の場合 :

Geant4が標準に持っている *G4UniformMagField* クラスを用いる

```
G4MagneticField* magField =  
    new G4UniformMagField{ G4ThreeVector(1.*Tesla,0.,0.) };
```

- 非一様磁場の場合 :

G4MagneticField クラスを継承して独自の磁場クラスを定義し、その *GetFieldValue* メソッドを実装する

```
class MyField : G4MagneticField {.....};  
  
.....  
void MyField::GetFieldValue(  
    const G4double Point[4], G4double *field) const {  
  
.....  
}
```

[注]

- *Point*[0..2] は空間点 (*global coordinate system*)、*Point*[3] は時間を表す
- *field*[0..2] は空間点に対応する磁場の返り値

Logical Volumeの定義 – 磁場の組み込み

- 磁場を表現しているG4MagneticFieldオブジェクトを構造体定義に組み込むには以下の二つの手法がある:

- 空間全体(World Volume)に対して組み込む — Global fieldの定義
- 構造体中の特定の物体内部に組み込む — Local fieldsの定義

[注] local fieldとglobal fieldは共存できる。共存させるとlocal fieldが優先

■ Global Fieldの定義方法

[手法1] G4TransportationManagerが独自に保持しているG4FieldManagerに、G4MagneticFieldオブジェクトを渡す

```
G4FieldManager* globalFieldMgr =  
    G4TransportationManager::GetTransportationManager()->GetFieldManager();  
globalFieldMgr->SetDetectorField(magField);  
globalFieldMgr->CreateChordFinder(magField);
```

[手法2] G4GlobalMagFieldMessengerに磁場vectorを直接わたす (G4 Version 10.0以降)

```
#include "G4GlobalMagFieldMessenger.hh"
```

使用例はHands-Onで学ぶ

.....

```
auto magFieldVector = G4ThreeVector{ 0.0, 1.0*tesla, 0.0 };  
new G4GlobalMagFieldMessenger{ magFieldVector };
```

Logical Volumeの定義 – 磁場の組み込み – つづき

■ Local Fieldの定義方法

- 新たにG4FieldManagerを作る – コンストラクタにlocal fieldを表現するG4MagneticFieldオブジェクトを渡す
- 磁場を設定したいlogical volumeに上記のG4FieldManagerを渡す

```
G4FieldManager* localFieldMgr  
= new G4FieldManager{ magField };  
  
logicalVolume->setFieldManager(localFieldMgr, true);
```

[注]

- 構造物の中の任意の物体内に磁場を設定できる
- local fieldが設定されればglobal fieldがは無視される
- 上記のコードにある' setFieldManager()' の第二引数の意味
 - true: 無条件に子物体に対しても定義された磁場伝搬する
 - false: 'G4FieldManager'を持っていない子物体のみに磁場が伝搬

講義 終了